ΜΙΑ ΠΡΩΤΗ ΑΡΧΙΚΗ ΓΝΩΡΙΜΙΑ ΜΕ ΤΗΝ ΙΔΙΑΙΤΕΡΗ «ΑΡΓΚΟ» ΤΗΣ C++

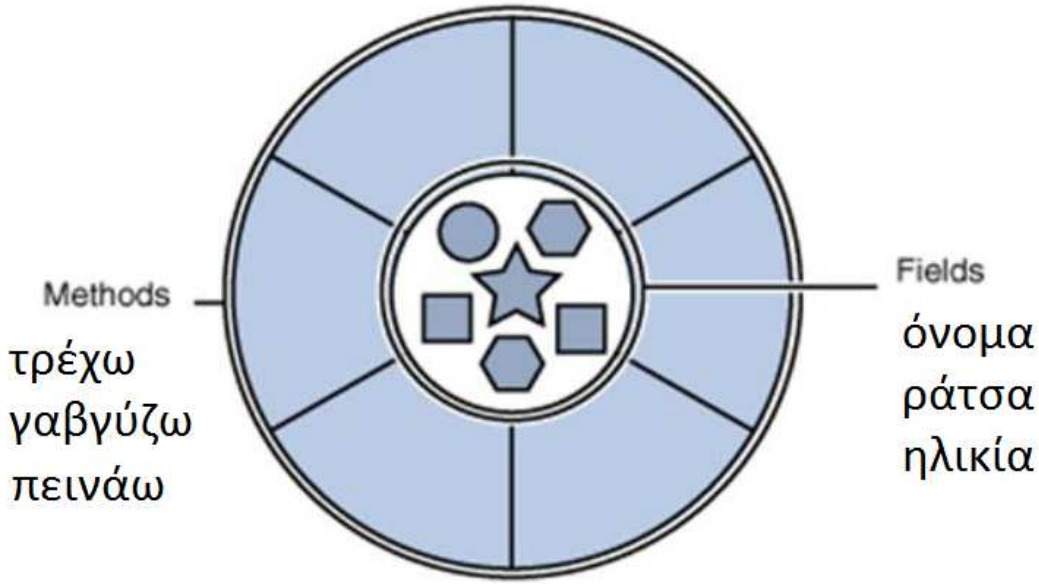
- Η καταννόηση της έννοιας του **αντικειμένου (object)** είναι βασική για την καταννόηση του **αντικειμενοστραφούς προγραμματισμού (object-oriented programming)**.

- Αντικείμενα υπάρχουν παντού στον πραγματικό κόσμο: θρανία, τηλεοράσεις, αυτοκίνητα, κτλ. Στον αντικειμενοστραφή προγραμματισμό ένα αντικείμενο δεν διαφέρει σχεδόν καθόλου με την θεώρηση στον πραγματικό κόσμο

- Κάθε αντικείμενο διαθέτει τα εξής:
  - Τα **πεδία (fields)** ή **χαρακτηριστικά (attributes)** του.
  - Τις **μεθόδους (methods)** του που αποτελούν τις ενέργειες που μπορεί να εκτελέσει.

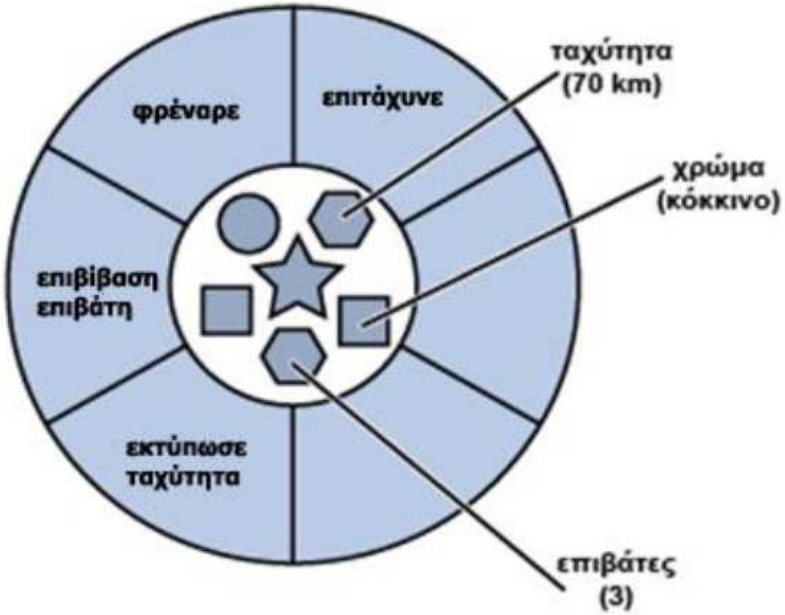
- Για παράδειγμα το αντικείμενο με όνομα «Rocky», το οποίο είναι σκυλάκι, έχει ως πεδία το όνομά του (που είναι «Rocky»), το χρώμα του (άσπρο) και την ηλικία του (5 ετών) ενώ ως μεθόδους τις ενέργειες «γάβγισμα», «τρέξιμο» και «κούνημα ουράς».

- Μπορούμε να φανταστούμε ένα αντικείμενο ως ένα «μπαλάκι» το οποίο έχει στο κέντρο/εσωτερικό του τα πεδία του και γύρω από αυτό τις μεθόδους του.



- Βασική επιδίωξη του αντικειμενοστραφούς προγραμματισμού, είναι η **επικοινωνία** με κάποιο αντικείμενο να γίνεται μόνο μέσω των μεθόδων του κι όχι με άμεση πρόσβαση στα πεδία του.

- Με τον όρο **επικοινωνία** αναφερόμαστε στα εξής:
  - όταν ζητάμε από το αντικείμενο να μεταβάλλει τις τιμές κάποιων πεδίων του.
  - όταν ζητάμε από το αντικείμενο να πληροφορηθούμε (π.χ. να μας εκτυπώσει) τις τιμές κάποιων πεδίων του.



- Στον πραγματικό κόσμο, μπορούμε να βρούμε πολλά αντικείμενα, τα οποία είναι του ίδιου τύπου. Για παράδειγμα, με μία απλή ματιά στον δρόμο του σπιτιού μας μπορούμε να δούμε πολλά αυτοκίνητα, δηλαδή πολλά αντικείμενα του τύπου «Αυτοκίνητο».

- Αυτό που διαφοροποιεί ουσιαστικά το κάθε αντικείμενο ενός τύπου από όλα τα υπόλοιπα, είναι οι τιμές που έχει τα πεδία του (θα μπορούσαν όμως να υπάρξουν και δύο διαφορετικά αντικείμενα με ακριβώς ίδιες τιμές σε όλα τα πεδία τους).

- Στον αντικειμενοστραφή προγραμματισμό, ονομάζουμε τον κάθε τύπο αντικειμένων ως **κλάση (class)** και λέμε ότι κάθε αντικείμενο αποτελεί ένα **στιγμιότυπο (instance)** της κλάσης στην οποία ανήκει.

- με τον όρο **μέθοδο (method)** ονομάζουμε ένα αυτόνομο κομμάτι κώδικα (συνώνυμα: υπορουτίνα, συνάρτηση).

- στις περισσότερες γλώσσες αντικειμενοστραφούς προγραμματισμού μπορούμε να αναγκάσουμε μία κλάση να έχει κάποιες μεθόδους.

- Η κάθε κλάση περιέχει μηδέν ή περισσότερες **μεθόδους (methods)**.

- Αυτό γίνεται ορίζοντας μία **διεπαφή (interface)** η οποία περιέχει τις υπογραφές των μεθόδων που επιθυμούμε η κλάση να έχει.

- Λέμε ότι μία κλάση **υλοποιεί (implements)** μία διεπαφή όταν **υλοποιεί (δηλαδή προσθέτει σώμα)** σε όλες τις μεθόδους της διεπαφής.

- Αν μία κλάση υλοποιεί μία διεπαφή, τότε είναι **υποχρεωτική** η προσθήκη σώματος σε όλες τις μεθόδους της διεπαφής.

- Κάθε αντικειμενοστραφής γλώσσα προγραμματισμού υποστηρίζει
  - 1. Ενθυλάκωση (Encapsulation)
  - 2. Αφαίρεση (Abstraction)
  - 3. Κληρονομικότητα (Inheritance)
  - 4. Πολυμορφισμός (Polymorphism)

- **1. Ενθυλάκωση (Encapsulation)**

- Σύμφωνα με την έννοια αυτή, αποκρύπτουμε από τον χρήστη των αντικειμένων της κλάσης, τα πεδία αυτής, με αποτέλεσμα ο χρήστης να μπορεί να επικοινωνεί μόνο μέσω της διεπαφής της (δηλαδή μέσω των μεθόδων της).

- Για παράδειγμα ο χρήστης για να αυξήσει την ταχύτητα ενός αντικειμένου τύπου «Αυτοκίνητο» καλεί την μέθοδο «επιτάχυνε».

- Ο χρήστης βλέπει την ταχύτητα να αυξάνεται, αλλά αυτό που στην πραγματικότητα κάνει η μέθοδος, είναι να προσθέτει έναν θετικό αριθμό στο πεδίο «ταχύτητα» του εν λόγω αντικειμένου.

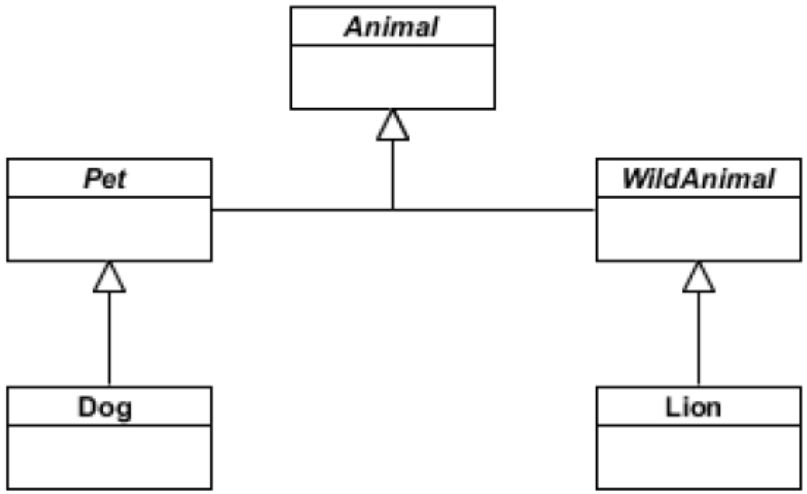
- Ο χρήστης δεν γνωρίζει τα πεδία που είναι αποθηκευμένα εσωτερικά στο αντικείμενο· βλέπει μόνο τις μεθόδους του και επικοινωνεί αποκλειστικά με αυτές.

- **2. Αφαίρεση (Abstraction)**
- Ο όρος αυτός αναφέρεται στα πεδία και τις μεθόδους μίας κλάσης τα οποία θα πρέπει να υλοποιήσει ένας προγραμματιστής.
- Αν σε ένα πρόγραμμα κάποιος θα πρέπει να δημιουργήσει την κλάση «Αυτοκίνητο», θα πρέπει να αποφασίσει ποια χαρακτηριστικά (πεδία) και ενέργειες (μεθόδου) είναι στην πραγματικότητα χρήσιμα.
- Στην ουσία ο προγραμματιστής λειτουργεί αφαιρετικά, υλοποιώντας μόνο ότι είναι απαραίτητο για την λειτουργία του προγράμματος.

➢ 3. Κληρονομικότητα (Inheritance)

➢ Η έννοια της **κληρονομικότητας (inheritance)** αποτελεί ένα από τα βασικότερα στοιχεία του αντικειμενοστραφούς προγραμματισμού. Όπως και στον πραγματικό κόσμο, κάποιες κλάσεις αποτελούν υπό-κατηγορίες άλλων κλάσεων.

➢ Για να είμαστε απολύτως ακριβείς με την ορολογία που χρησιμοποιείται στον αντικειμενοστραφή προγραμματισμό, ονομάζουμε την κλάση που βρίσκεται παραπάνω στην ιεραρχία που δημιουργείται από μία κληρονομικότητα **υπέρ-κλάση (super class)** ενώ την κλάση που βρίσκεται παρακάτω **υπό-κλάση (sub class)**.



- **4. Πολυμορφισμός (Polymorphism)**
- Πολυμορφισμός τυπικά σημαίνει: «ένα όνομα, πολλές μορφές» (“one name, many forms”).
- Ουσιαστικά πολυμορφισμός είναι η δυνατότητα δημιουργίας πολλών μεθόδων με το ίδιο όνομα, οι οποίες όμως έχουν διαφορετική συμπεριφορά.
- Υπάρχουν δύο είδη πολυμορφισμού: ο στατικός πολυμορφισμός (static polymorphism) και ο δυναμικός πολυμορφισμός (dynamic polymorphism).
- **Στατικός πολυμορφισμός:** η δυνατότητα ορισμού πολλών μεθόδων με το ίδιο όνομα στην ίδια κλάση. Η κάθε μέθοδος διαφέρει από τις άλλες στα ορίσματα που δέχεται.
- **Δυναμικός πολυμορφισμός:** η δυνατότητα επανορισμού του σώματος μιας μεθόδου που κληρονομείται. Μία υπό-κλάση η οποία κληρονομεί μία μέθοδο από μία υπέρ-κλάση, μπορεί να αλλάξει το σώμα της μεθόδου και να την προσαρμόσει ανάλογα με τις ανάγκες της.